Document number: P2501R0
Audience: LEWG

Ville Voutilainen
2021-12-09

Undo the rename of views::move and views::as_const

Abstract

This paper seeks to undo the rename of views::move to views::all_move, and the rename of views::as_const to views::all_const. The renames make no sense, and are based on an unsubstantiated knee-jerk reaction, creating a situation where the new names are inconsistent and incorrect, and don't fit into the design intent of our range views.

The alleged confusion

The suggested rationale for the rename was that it's confusing to have a views::move and a std::move (two of them), and a views::as_const and a std::as_const. It was even suggested that that's confusing for "everybody".

A further explanation was that it's allegedly confusing whether a views::move or a views::as_const applies to the range or view provided as the argument, or to the elements.

There is no confusion, or there at least shouldn't be

Both of these suggested confusions are non-sensical. First of all,

1. views are (lazy) algorithms.
2. those algorithms, like all other algorithms, always apply to the elements.

This includes algorithms that look like they apply to a range or view; an example is join (or join_with). Some might claim that it joins ranges. But it doesnt, it joins the elements of the ranges. It merely creates an algorithm that allows viewing the elements as a flattened view. Whether it somewhere internally somehow does that by joining ranges or views is an implementation detail, not the API-level conceptual functionality provided by the view.

And you can see that the joining of the inner things is an implementation detail; all that is exposition-only. What join gives you is the joined view of the elements. It joins elements.

There was a suggestion that filter is somehow another view that filters a range. Sure, it does - by filtering elements. The range is there just as a vessel for the elements, and the filtering algorithm applies to them, not to the range. The range is there only as an exposition-only implementation detail, and the predicate applies to an element.

It's the same thing for views::move and views::as_const. The first one gives you a view of the elements cast to rvalue references, i.e. a view of the moved elements. Like applying std::move on each element. The second one gives you a view of the elements with as_const applied to each element.

And that's why views::move and views::as_const are the *CORRECT* names for these views. They say what the view does on the tin. They are views that apply a std::move or a std::as_const to their elements. *Of course* they are named views::move and views::as_const, the similarity of those names is 100% intentional, because those are the operations that the views apply!

As it's explained in p2278r1.html#naming, "The advantage of views::as_const is that it is a mirror of std::as_const and so sharing a name seems reasonable."

That's more than just reasonable. It's the right name. Yes, the paper admits that it's possible to manage to misuse std::as_const where you want to use std::ranges::views::as_const. But such possible misuses don't change the _fact_ that the view mirrors std::as_const, and applies effectively std::as_const to every element. The same goes for views::move; it applies std::move to every element.

All of this is intentional. The name of the lazy algorithm tells its users what operation (transformation) it applies to the elements it lazily operates on.

The suggested alternatives, views::all_move and views::all_const don't do that. They are themselves confusing, in a far worse way than the correct names are. "What do you mean 'all'? The view applies to all of the elements already, why are you saying 'all', that makes no sense." And they don't communicate what transform they apply on the elements. They're now lazy algorithms with completely weird, inconsistent, and inexplicable names. The only explanation for their names would be "we wanted to make them different from the operations they perform, because.. uh.. yeah." I can't explain that. The original names are perfectly explicable, this is how range views work, this is what they do. The renamed names require an out-of-the-blue explanation that doesn't ride on the design principles of our range views, they're completely hacked names, renamed on a whim, with no real field usage behind them. That makes them design-wise inconsistent names, and they should not be.

The solution

This is simple: undo the rename, and ship views::move as views::move, and ship views::as_const as views::as_const.